

Thiết kế Exception Handling chuẩn Production cho Spring Boot

1. Mục tiêu

Trong hệ thống Spring Boot production, exception handling không chỉ là `try-catch` để tránh crash ứng dụng. Một thiết kế chuẩn cần đạt các mục tiêu:

- Trả response lỗi nhất quán cho client.
- Không expose stack trace, SQL error, internal class name ra ngoài.
- Có `errorCode` rõ ràng để frontend/mobile xử lý.
- Có `traceId/requestId` để truy vết log khi production incident.
- Phân loại được lỗi business, validation, authentication, authorization, database, network, third-party service.
- Logging đúng mức độ, không log dữ liệu nhạy cảm.
- Hỗ trợ monitoring/alerting qua log, metrics, tracing.
- Dễ mở rộng khi hệ thống lớn dần hoặc chuyển sang microservices.

2. Nguyên tắc thiết kế

2.1. Không trả exception thô cho client

Không nên để Spring Boot trả lỗi mặc định kiểu:

```
{
  "timestamp": "2026-06-04T10:00:00",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "...",
  "path": "/api/orders"
}
```

Vì response này:

- Không ổn định cho frontend.
- Có thể lộ stack trace.
- Không có business error code.
- Khó debug nếu không có `traceId`.

Nên chuẩn hóa response lỗi.

2.2. Phân biệt lỗi business và lỗi system

Một hệ thống production thường chia exception thành các nhóm:

Nhóm lỗi	Ví dụ	HTTP Status
Validation Error	thiếu field, sai format email	400
Business Error	số dư không đủ, order đã thanh toán	400 / 409
Not Found	không tìm thấy user/order	404
Unauthorized	chưa đăng nhập, token sai	401
Forbidden	không có quyền	403
Conflict	duplicate resource, version conflict	409
External Service Error	payment gateway lỗi, shipping API timeout	502 / 503 / 504
Database Error	lỗi connection, constraint, transaction	500 / 409
Internal Error	bug không mong muốn	500

2.3. Không dùng exception để điều khiển flow thông thường

Không nên viết:

```
try {  
    userRepository.findById(id).get();  
} catch (NoSuchElementException e) {  
    throw new UserNotFoundException(id);  
}
```

Nên viết rõ ràng:

```
User user = userRepository.findById(id)  
    .orElseThrow(() -> new UserNotFoundException(id));
```

2.4. Không catch Exception quá rộng ở service nếu không cần

Không nên:

```
try {
    paymentClient.charge(request);
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

Vì mất thông tin lỗi gốc và khó phân loại.

Nên catch theo nhóm cụ thể:

```
try {
    paymentClient.charge(request);
} catch (SocketTimeoutException e) {
    throw new ExternalServiceTimeoutException("PAYMENT_SERVICE", e);
} catch (IOException e) {
    throw new ExternalServiceException("PAYMENT_SERVICE", e);
}
```

3. Chuẩn response lỗi

Một format response phổ biến trong production:

```
{
  "timestamp": "2026-06-04T10:15:30Z",
  "status": 400,
  "error": "BAD_REQUEST",
  "code": "ORDER_001",
  "message": "Order has already been paid",
  "path": "/api/orders/1001/pay",
  "traceId": "4f8c2a1d9b234f7a",
  "details": [
    {
      "field": "quantity",
      "message": "must be greater than 0"
    }
  ]
}
```

Ý nghĩa:

Field	Ý nghĩa
timestamp	thời điểm lỗi xảy ra
status	HTTP status code
error	tên HTTP error
code	mã lỗi nội bộ
message	thông báo an toàn cho client
path	API path
traceId	ID để trace log
details	lỗi chi tiết, thường dùng cho validation

4. Cấu trúc package đề xuất

```
src/main/java/com/example/project
├── common
│   ├── exception
│   │   ├── ErrorCode.java
│   │   ├── BaseException.java
│   │   ├── BusinessException.java
│   │   ├── ResourceNotFoundException.java
│   │   ├── ExternalServiceException.java
│   │   ├── ExternalServiceTimeoutException.java
│   │   └── GlobalExceptionHandler.java
│   ├── response
│   │   ├── ErrorResponse.java
│   │   └── FieldErrorResponse.java
│   └── tracing
│       └── RequestIdFilter.java
├── order
│   ├── exception
│   │   ├── OrderNotFoundException.java
│   │   └── OrderAlreadyPaidException.java
│   └── service
└── payment
```

└─ exception
└─ service

5. Định nghĩa ErrorCode

```
package com.example.common.exception;

import org.springframework.http.HttpStatus;

public enum ErrorCode {

    // Common
    INTERNAL_SERVER_ERROR("COMMON_500", "Internal server error",
        HttpStatus.INTERNAL_SERVER_ERROR),
    INVALID_REQUEST("COMMON_400", "Invalid request", HttpStatus.BAD_REQUEST),
    VALIDATION_ERROR("COMMON_422", "Validation failed",
        HttpStatus.UNPROCESSABLE_ENTITY),
    RESOURCE_NOT_FOUND("COMMON_404", "Resource not found",
        HttpStatus.NOT_FOUND),
    UNAUTHORIZED("COMMON_401", "Unauthorized", HttpStatus.UNAUTHORIZED),
    FORBIDDEN("COMMON_403", "Forbidden", HttpStatus.FORBIDDEN),
    CONFLICT("COMMON_409", "Conflict", HttpStatus.CONFLICT),

    // Order
    ORDER_NOT_FOUND("ORDER_404", "Order not found", HttpStatus.NOT_FOUND),
    ORDER_ALREADY_PAID("ORDER_001", "Order has already been paid",
        HttpStatus.CONFLICT),
    ORDER_CANNOT_BE_CANCELLED("ORDER_002", "Order cannot be cancelled",
        HttpStatus.BAD_REQUEST),

    // Payment
    PAYMENT_FAILED("PAYMENT_001", "Payment failed", HttpStatus.BAD_REQUEST),
    PAYMENT_TIMEOUT("PAYMENT_504", "Payment service timeout",
        HttpStatus.GATEWAY_TIMEOUT),
    PAYMENT_SERVICE_UNAVAILABLE("PAYMENT_503", "Payment service unavailable",
        HttpStatus.SERVICE_UNAVAILABLE),

    // External
    EXTERNAL_SERVICE_ERROR("EXT_001", "External service error",
        HttpStatus.BAD_GATEWAY),
    EXTERNAL_SERVICE_TIMEOUT("EXT_504", "External service timeout",
        HttpStatus.GATEWAY_TIMEOUT),

    // Database
```

```

        DATABASE_ERROR("DB_500", "Database error",
HttpStatus.INTERNAL_SERVER_ERROR),
        DUPLICATE_RESOURCE("DB_409", "Duplicate resource", HttpStatus.CONFLICT);

private final String code;
private final String defaultMessage;
private final HttpStatus status;

ErrorCode(String code, String defaultMessage, HttpStatus status) {
    this.code = code;
    this.defaultMessage = defaultMessage;
    this.status = status;
}

public String getCode() {
    return code;
}

public String getDefaultMessage() {
    return defaultMessage;
}

public HttpStatus getStatus() {
    return status;
}
}

```

6. BaseException

```

package com.example.common.exception;

public abstract class BaseException extends RuntimeException {

    private final ErrorCode errorCode;

    protected BaseException(ErrorCode errorCode) {
        super(errorCode.getDefaultMessage());
        this.errorCode = errorCode;
    }

    protected BaseException(ErrorCode errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
    }
}

```

```
    protected BaseException(ErrorCode errorCode, String message, Throwable
cause) {
        super(message, cause);
        this.errorCode = errorCode;
    }

    protected BaseException(ErrorCode errorCode, Throwable cause) {
        super(errorCode.getDefaultMessage(), cause);
        this.errorCode = errorCode;
    }

    public ErrorCode getErrorCode() {
        return errorCode;
    }
}
```

7. BusinessException

```
package com.example.common.exception;

public class BusinessException extends BaseException {

    public BusinessException(ErrorCode errorCode) {
        super(errorCode);
    }

    public BusinessException(ErrorCode errorCode, String message) {
        super(errorCode, message);
    }

    public BusinessException(ErrorCode errorCode, String message, Throwable
cause) {
        super(errorCode, message, cause);
    }
}
```

8. ResourceNotFoundException

```
package com.example.common.exception;
```

```

public class ResourceNotFoundException extends BaseException {

    public ResourceNotFoundException(ErrorCode errorCode) {
        super(errorCode);
    }

    public ResourceNotFoundException(ErrorCode errorCode, String message) {
        super(errorCode, message);
    }
}

```

Ví dụ exception riêng cho Order:

```

package com.example.order.exception;

import com.example.common.exception.ErrorCode;
import com.example.common.exception.ResourceNotFoundException;

public class OrderNotFoundException extends ResourceNotFoundException {

    public OrderNotFoundException(Long orderId) {
        super(ErrorCode.ORDER_NOT_FOUND, "Order not found with id: " + orderId);
    }
}

```

9. ExternalServiceException

Dùng cho lỗi khi gọi service bên ngoài như Payment Gateway, Shipping API, Email Provider, OneSignal, Elasticsearch, Kafka REST Proxy.

```

package com.example.common.exception;

public class ExternalServiceException extends BaseException {

    private final String serviceName;

    public ExternalServiceException(String serviceName, Throwable cause) {
        super(
            ErrorCode.EXTERNAL_SERVICE_ERROR,
            "External service error: " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }
}

```



```

    }

    public ExternalServiceException(ErrorCode errorCode, String serviceName,
    Throwable cause) {
        super(errorCode, errorCode.getDefaultMessage() + ": " + serviceName,
        cause);
        this.serviceName = serviceName;
    }

    public String getServiceName() {
        return serviceName;
    }
}

```

10. ExternalServiceTimeoutException

```

package com.example.common.exception;

public class ExternalServiceTimeoutException extends BaseException {

    private final String serviceName;

    public ExternalServiceTimeoutException(String serviceName, Throwable cause)
    {
        super(
            ErrorCode.EXTERNAL_SERVICE_TIMEOUT,
            "External service timeout: " + serviceName,
            cause
        );
        this.serviceName = serviceName;
    }

    public String getServiceName() {
        return serviceName;
    }
}

```

11. ErrorResponse DTO

```

package com.example.common.response;

```

```

import com.fasterxml.jackson.annotation.JsonInclude;

import java.time.Instant;
import java.util.List;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private Instant timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private String path;
    private String traceId;
    private List<FieldErrorResponse> details;

    public ErrorResponse() {
    }

    public ErrorResponse(
        Instant timestamp,
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId,
        List<FieldErrorResponse> details
    ) {
        this.timestamp = timestamp;
        this.status = status;
        this.error = error;
        this.code = code;
        this.message = message;
        this.path = path;
        this.traceId = traceId;
        this.details = details;
    }

    public static ErrorResponse of(
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId
    ) {

```

```

        return new ErrorResponse(
            Instant.now(),
            status,
            error,
            code,
            message,
            path,
            traceId,
            null
        );
    }

    public static ErrorResponse of(
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId,
        List<FieldErrorResponse> details
    ) {
        return new ErrorResponse(
            Instant.now(),
            status,
            error,
            code,
            message,
            path,
            traceId,
            details
        );
    }

    // Getter/setter omitted for brevity
}

```

Field error DTO:

```

package com.example.common.response;

public class FieldErrorResponse {

    private String field;
    private String message;
    private Object rejectedValue;
}

```

```

    public FieldErrorResponse() {
    }

    public FieldErrorResponse(String field, String message, Object
rejectedValue) {
        this.field = field;
        this.message = message;
        this.rejectedValue = rejectedValue;
    }

    public String getField() {
        return field;
    }

    public String getMessage() {
        return message;
    }

    public Object getRejectedValue() {
        return rejectedValue;
    }
}

```

12. GlobalExceptionHandler

```

package com.example.common.exception;

import com.example.common.response.ErrorResponse;
import com.example.common.response.FieldErrorResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.ConstraintViolationException;
import org.slf4j.MDC;
import org.springframework.dao.DataAccessException;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.HttpRequestMethodNotSupportedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;

```

```

import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.io.IOException;
import java.util.List;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final String TRACE_ID = "traceId";

    @ExceptionHandler(BaseException.class)
    public ResponseEntity<ErrorResponse> handleBaseException(
        BaseException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ex.getErrorCode();
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            ex.getMessage(),
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationException(
        MethodArgumentNotValidException ex,
        HttpServletRequest request
    ) {
        List<FieldErrorResponse> details = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(this::toFieldErrorResponse)
            .toList();

        ErrorCode errorCode = ErrorCode.VALIDATION_ERROR;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),

```

```

        errorCode.getCode(),
        errorCode.getDefaultMessage(),
        request.getRequestURI(),
        getTraceId(),
        details
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(ConstraintViolationException.class)
public ResponseEntity<ErrorResponse> handleConstraintViolationException(
    ConstraintViolationException ex,
    HttpServletRequest request
) {
    List<FieldErrorResponse> details = ex.getConstraintViolations()
        .stream()
        .map(violation -> new FieldErrorResponse(
            violation.getPropertyPath().toString(),
            violation.getMessage(),
            violation.getInvalidValue()
        ))
        .toList();

    ErrorCode errorCode = ErrorCode.VALIDATION_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        errorCode.getDefaultMessage(),
        request.getRequestURI(),
        getTraceId(),
        details
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponse> handleInvalidJson(
    HttpMessageNotReadableException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.INVALID_REQUEST;
    HttpStatus status = errorCode.getStatus();

```

```

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Malformed JSON request",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<ErrorResponse> handleMissingParameter(
        MissingServletRequestParameterException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.INVALID_REQUEST;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Missing required parameter: " + ex.getParameterName(),
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(HttpRequestMethodNotSupportedException.class)
    public ResponseEntity<ErrorResponse> handleMethodNotSupported(
        HttpRequestMethodNotSupportedException ex,
        HttpServletRequest request
    ) {
        HttpStatus status = HttpStatus.METHOD_NOT_ALLOWED;

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            "COMMON_405",
            "HTTP method not supported",
            request.getRequestURI(),
            getTraceId()
        );
    }

```

```

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ResponseEntity<ErrorResponse> handleBadCredentials(
        BadCredentialsException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.UNAUTHORIZED;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Invalid username or password",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handleAccessDenied(
        AccessDeniedException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.FORBIDDEN;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Access denied",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleDataIntegrityViolation(
        DataIntegrityViolationException ex,
        HttpServletRequest request
    ) {

```



```

        ErrorCode errorCode = ErrorCode.DUPLICATE_RESOURCE;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Resource already exists or violates database constraint",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    public ResponseEntity<ErrorResponse> handleOptimisticLocking(
        ObjectOptimisticLockingFailureException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.CONFLICT;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            "Resource was modified by another transaction. Please retry",
            request.getRequestURI(),
            getTraceId()
        );

        return ResponseEntity.status(status).body(response);
    }

    @ExceptionHandler(DataAccessException.class)
    public ResponseEntity<ErrorResponse> handleDatabaseException(
        DataAccessException ex,
        HttpServletRequest request
    ) {
        ErrorCode errorCode = ErrorCode.DATABASE_ERROR;
        HttpStatus status = errorCode.getStatus();

        ErrorResponse response = ErrorResponse.of(
            status.value(),
            status.name(),
            errorCode.getCode(),
            errorCode.getDefaultMessage(),

```

```

        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(IOException.class)
public ResponseEntity<ErrorResponse> handleIOException(
    IOException ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.EXTERNAL_SERVICE_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        "I/O error while processing request",
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse> handleUnexpectedException(
    Exception ex,
    HttpServletRequest request
) {
    ErrorCode errorCode = ErrorCode.INTERNAL_SERVER_ERROR;
    HttpStatus status = errorCode.getStatus();

    ErrorResponse response = ErrorResponse.of(
        status.value(),
        status.name(),
        errorCode.getCode(),
        errorCode.getDefaultMessage(),
        request.getRequestURI(),
        getTraceId()
    );

    return ResponseEntity.status(status).body(response);
}

private FieldErrorResponse toFieldErrorResponse(FieldError fieldError) {

```

```

        return new FieldErrorResponse(
            fieldError.getField(),
            fieldError.getDefaultMessage(),
            fieldError.getRejectedValue()
        );
    }

    private String getTraceId() {
        return MDC.get	TRACE_ID);
    }
}

```

13. RequestIdFilter / TraceIdFilter

Nếu chưa dùng distributed tracing như Micrometer Tracing, OpenTelemetry, Zipkin, Jaeger, có thể tự tạo traceId đơn giản.

```

package com.example.common.tracing;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.UUID;

@Component
public class RequestIdFilter extends OncePerRequestFilter {

    private static final String TRACE_ID = "traceId";
    private static final String TRACE_ID_HEADER = "X-Trace-Id";

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        String traceId = request.getHeader(TRACE_ID_HEADER);
    }
}

```

```

        if (traceId == null || traceId.isBlank()) {
            traceId = UUID.randomUUID().toString();
        }

        try {
            MDC.put	TRACE_ID, traceId);
            response.setHeader	TRACE_ID_HEADER, traceId);
            filterChain.doFilter(request, response);
        } finally {
            MDC.remove	TRACE_ID);
        }
    }
}

```

Logback pattern:

```

<pattern>
    %d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level traceId=%X{traceId}
%logger{36} - %msg%n
</pattern>

```

14. Ví dụ dùng trong Service Layer

14.1. Not Found

```

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderResponse getOrder(Long orderId) {
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));

        return OrderResponse.from(order);
    }
}

```

14.2. Business Rule

```
public void payOrder(Long orderId) {
    Order order = orderRepository.findById(orderId)
        .orElseThrow(() -> new OrderNotFoundException(orderId));

    if (order.isPaid()) {
        throw new BusinessException(
            ErrorCode.ORDER_ALREADY_PAID,
            "Order " + orderId + " has already been paid"
        );
    }

    // continue payment flow
}
```

14.3. IOException khi gọi external service

Ví dụ gọi service bên ngoài bằng `WebClient`, `RestTemplate`, SDK hoặc thao tác file.

```
public void uploadInvoice(Long orderId) {
    try {
        invoiceStorageClient.upload(orderId);
    } catch (SocketTimeoutException e) {
        throw new ExternalServiceTimeoutException("INVOICE_STORAGE", e);
    } catch (IOException e) {
        throw new ExternalServiceException("INVOICE_STORAGE", e);
    }
}
```

Nguyên tắc:

- Không trả trực tiếp message từ `IOException` cho client.
- Log chi tiết ở server.
- Client chỉ nhận message an toàn.
- Nếu lỗi do timeout, nên map sang 504.
- Nếu lỗi do service bên ngoài unavailable, nên map sang 503 hoặc 502.

15. Exception handling cho Validation

Request DTO:

```

public class CreateOrderRequest {

    @NotNull(message = "userId is required")
    private Long userId;

    @NotEmpty(message = "items must not be empty")
    private List<OrderItemRequest> items;

    @NotBlank(message = "shippingAddress is required")
    private String shippingAddress;

    // getter/setter
}

```

Controller:

```

@PostMapping("/orders")
public ResponseEntity<OrderResponse> createOrder(
    @Valid @RequestBody CreateOrderRequest request
) {
    OrderResponse response = orderService.createOrder(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}

```

Response l i validation:

```

{
  "timestamp": "2026-06-04T10:15:30Z",
  "status": 422,
  "error": "UNPROCESSABLE_ENTITY",
  "code": "COMMON_422",
  "message": "Validation failed",
  "path": "/api/orders",
  "traceId": "8f5d2f15-622b-48e2-91a9-6b08b3f14a2e",
  "details": [
    {
      "field": "userId",
      "message": "userId is required",
      "rejectedValue": null
    },
    {
      "field": "items",
      "message": "items must not be empty",
      "rejectedValue": []
    }
  ]
}

```

```
}  
]  
}
```

16. Exception handling cho Spring Security

Với Spring Security, một số exception xảy ra trước khi request vào controller, nên `@RestControllerAdvice` có thể không bắt được đầy đủ.

Nên custom:

- `AuthenticationEntryPoint` cho lỗi 401.
- `AccessDeniedHandler` cho lỗi 403.

16.1. CustomAuthenticationEntryPoint

```
@Component  
public class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint  
{  
  
    private final ObjectMapper objectMapper;  
  
    public CustomAuthenticationEntryPoint(ObjectMapper objectMapper) {  
        this.objectMapper = objectMapper;  
    }  
  
    @Override  
    public void commence(  
        HttpServletRequest request,  
        HttpServletResponse response,  
        AuthenticationException authException  
    ) throws IOException {  
  
        ErrorResponse errorResponse = ErrorResponse.of(  
            HttpStatus.UNAUTHORIZED.value(),  
            HttpStatus.UNAUTHORIZED.name(),  
            ErrorCode.UNAUTHORIZED.getCode(),  
            "Unauthorized",  
            request.getRequestURI(),  
            MDC.get("traceId")  
        );  
  
        response.setStatus(HttpStatus.UNAUTHORIZED.value());  
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);  
    }  
}
```

```

        objectMapper.writeValue(response.getWriter(), errorResponse);
    }
}

```

16.2. CustomAccessDeniedHandler

```

@Component
public class CustomAccessDeniedHandler implements AccessDeniedHandler {

    private final ObjectMapper objectMapper;

    public CustomAccessDeniedHandler(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
    }

    @Override
    public void handle(
        HttpServletRequest request,
        HttpServletResponse response,
        AccessDeniedException accessDeniedException
    ) throws IOException {

        ErrorResponse errorResponse = ErrorResponse.of(
            HttpStatus.FORBIDDEN.value(),
            HttpStatus.FORBIDDEN.name(),
            ErrorCode.FORBIDDEN.getCode(),
            "Access denied",
            request.getRequestURI(),
            MDC.get("traceId")
        );

        response.setStatus(HttpStatus.FORBIDDEN.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        objectMapper.writeValue(response.getWriter(), errorResponse);
    }
}

```

16.3. SecurityConfig

```

@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final CustomAuthenticationEntryPoint authenticationEntryPoint;
}

```



```

private final CustomAccessDeniedHandler accessDeniedHandler;

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    return http
        .csrf(AbstractHttpConfigurer::disable)
        .exceptionHandling(exception -> exception
            .authenticationEntryPoint(authenticationEntryPoint)
            .accessDeniedHandler(accessDeniedHandler)
        )
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
        )
        .build();
}
}

```

17. Exception handling khi gọi service bên ngoài

17.1. Với WebClient

```

public PaymentResponse charge(PaymentRequest request) {
    try {
        return webClient.post()
            .uri("/payments/charge")
            .bodyValue(request)
            .retrieve()
            .onStatus(
                HttpStatusCode::is4xxClientError,
                response -> response.bodyToMono(String.class)
                    .map(body -> new BusinessException(
                        ErrorCode.PAYMENT_FAILED,
                        "Payment rejected by provider"
                    ))
            )
            .onStatus(
                HttpStatusCode::is5xxServerError,
                response -> response.bodyToMono(String.class)
                    .map(body -> new ExternalServiceException(
                        "PAYMENT_PROVIDER",
                        new RuntimeException(body)
                    ))
            )
    }
}

```

```

        )
        .bodyToMono(PaymentResponse.class)
        .block();

    } catch (WebClientRequestException e) {
        throw new ExternalServiceTimeoutException("PAYMENT_PROVIDER", e);
    }
}

```

17.2. Với OpenFeign

```

@Component
public class FeignErrorDecoder implements ErrorDecoder {

    private final ErrorDecoder defaultErrorDecoder = new Default();

    @Override
    public Exception decode(String methodKey, Response response) {
        int status = response.status();

        if (status == 400) {
            return new BusinessException(ErrorCode.PAYMENT_FAILED);
        }

        if (status == 404) {
            return new ResourceNotFoundException(
                ErrorCode.RESOURCE_NOT_FOUND,
                "Resource not found from external service"
            );
        }

        if (status == 503) {
            return new ExternalServiceException(
                ErrorCode.PAYMENT_SERVICE_UNAVAILABLE,
                "PAYMENT_SERVICE",
                null
            );
        }

        return defaultErrorDecoder.decode(methodKey, response);
    }
}

```

18. Logging strategy chuẩn production

18.1. Log gì?

Nên log:

- traceId/requestId
- userId nếu có
- endpoint
- HTTP method
- status code
- errorCode
- exception class
- message nội bộ
- external service name
- duration
- request metadata

Không nên log:

- password
- access token
- refresh token
- OTP
- CVV
- full card number
- private key
- authorization header
- sensitive request body

18.2. Logging level

Loại lỗi	Log level
Validation error	DEBUG hoặc INFO
Business error expected	INFO
Not found thông thường	INFO hoặc DEBUG
Unauthorized/Forbidden	WARN nếu bất thường
External service timeout	WARN
Database error	ERROR
Unexpected exception	ERROR
Security attack suspected	WARN hoặc ERROR

Ví dụ:

```
log.warn(  
    "External service timeout. service={}, traceId={}, message={}",  
    serviceName,  
    MDC.get("traceId"),  
    ex.getMessage(),  
    ex  
);
```

Với lỗi 500:

```
log.error(  
    "Unexpected error occurred. traceId={}, path={}, message={}",  
    MDC.get("traceId"),  
    request.getRequestURI(),  
    ex.getMessage(),  
    ex  
);
```

19. Có nên log trong GlobalExceptionHandler không?

Có, nhưng cần phân biệt.

Ví dụ:

```
@ExceptionHandler(BaseException.class)  
public ResponseEntity<ErrorResponse> handleBaseException(  
    BaseException ex,  
    HttpServletRequest request  
) {  
    HttpStatus status = ex.getErrorCode().getStatus();  
  
    if (status.is5xxServerError()) {  
        log.error("Application exception. code={}, path={}",  
            ex.getErrorCode().getCode(),  
            request.getRequestURI(),  
            ex);  
    } else {  
        log.info("Handled business exception. code={}, path={}, message={}",  
            ex.getErrorCode().getCode(),  
            request.getRequestURI(),  
            ex.getMessage());  
    }  
}
```

```

        ex.getMessage());
    }

    // build response
}

```

Không nên log ERROR cho mọi exception, vì validation/business error là lỗi expected. Nếu log ERROR quá nhiều, production alert sẽ bị nhiễu.

20. Transaction và exception

Trong Spring, `@Transactional` mặc định rollback với `RuntimeException` và `Error`, không rollback với checked exception.

Ví dụ:

```

@Transactional
public void placeOrder(PlaceOrderRequest request) {
    orderRepository.save(order);

    if (invalid) {
        throw new BusinessException(ErrorCode.ORDER_CANNOT_BE_CANCELLED);
    }
}

```

Vì `BusinessException` extends `RuntimeException`, transaction sẽ rollback.

Nếu dùng checked exception:

```

@Transactional(rollbackFor = IOException.class)
public void importFile(MultipartFile file) throws IOException {
    // process file
}

```

Trong production, nhiều team ưu tiên custom exception extends `RuntimeException` để transaction rollback tự nhiên hơn.

21. Exception trong async processing

Với `@Async`, exception không tự trả về client vì chạy ở thread khác.

Ví dụ:

```
@Async
public void sendEmailAsync(Long orderId) {
    emailService.sendInvoice(orderId);
}
```

Nếu `sendInvoice` throw exception, request chính có thể vẫn thành công.

Cần xử lý bằng:

- `AsyncUncaughtExceptionHandler`
- retry mechanism
- dead letter queue nếu dùng message broker
- outbox pattern nếu cần reliability cao

Ví dụ:

```
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) -> {
            log.error("Async method failed. method={}, params={}",
                method.getName(),
                Arrays.toString(params),
                ex);
        };
    }
}
```

22. Exception trong Kafka Consumer

Trong Kafka consumer, không nên để exception làm consumer loop chết liên tục.

Cần thiết kế:

- retry cho lỗi tạm thời
- dead letter topic cho lỗi không xử lý được
- idempotency để tránh xử lý trùng message
- log kèm eventId, correlationId

- phân biệt business error và technical error

Ví dụ:

```
@KafkaListener(topics = "order-created")
public void consume(OrderCreatedEvent event) {
    try {
        orderEventHandler.handle(event);
    } catch (BusinessException e) {
        log.warn("Business error while consuming event. eventId={}, code={}",
            event.getEventId(),
            e.getErrorCode().getCode());
    } catch (Exception e) {
        log.error("Failed to consume order-created event. eventId={}",
            event.getEventId(),
            e);
        throw e;
    }
}
```

Với Spring Kafka production nên cấu hình:

- DefaultErrorHandler
- DeadLetterPublishingRecoverer
- backoff retry
- DLT topic

23. Exception trong scheduled job

Scheduled job không có client nhận response. Vì vậy cần:

- log lỗi rõ ràng
- không để job chết âm thầm
- có alert nếu job fail
- lưu execution history nếu job quan trọng
- tránh duplicate nếu chạy nhiều instance bằng distributed lock

Ví dụ:

```
@Scheduled(cron = "0 0 * * * *")
public void sendPaymentReminder() {
    try {
        reminderService.sendPaymentReminders();
    } catch (Exception e) {

```

```

    log.error("Payment reminder job failed", e);
  }
}

```

Với production tốt hơn nên lưu bảng:

```

job_execution
- id
- job_name
- started_at
- finished_at
- status
- error_message
- trace_id

```

24. Mapping HTTP status chuẩn

Case	HTTP Status	Error Code ví dụ
Request body sai JSON	400	COMMON_400
Validation fail	422 hoặc 400	COMMON_422
Token thiếu/sai	401	COMMON_401
Không có quyền	403	COMMON_403
Không tìm thấy resource	404	ORDER_404
Duplicate key	409	DB_409
Optimistic locking conflict	409	COMMON_409
Business rule conflict	409	ORDER_001
External service lỗi	502	EXT_001
External service unavailable	503	EXT_503
External service timeout	504	EXT_504
Bug server	500	COMMON_500

25. Những lỗi thiết kế thường gặp

25.1. Trả message exception gốc cho client

Không nên:

```
return ex.getMessage();
```

Vì có thể lộ:

- SQL query
- table name
- column name
- file path
- internal URL
- secret
- stack trace

25.2. Dùng một RuntimeException cho mọi lỗi

Không nên:

```
throw new RuntimeException("Order not found");  
throw new RuntimeException("Payment failed");  
throw new RuntimeException("User not allowed");
```

Vì không phân loại được lỗi, khó map status code và error code.

25.3. Catch rồi nuốt exception

Không nên:

```
try {  
    paymentService.charge();  
} catch (Exception e) {  
    log.error("Payment failed");  
}
```

Vì hệ thống có thể tiếp tục chạy như thể payment đã thành công.

25.4. Log thiếu stack trace

Không nên:

```
log.error("Payment failed: {}", e.getMessage());
```

Nên:

```
log.error("Payment failed", e);
```

25.5. Log dữ liệu nhạy cảm

Không nên:

```
log.info("Login request: {}", loginRequest);
```

Nếu `loginRequest` có password, token, OTP thì rất nguy hiểm.

26. Checklist production

API response

- Có format lỗi thống nhất.
- Có error code.
- Có message an toàn.
- Có traceId.
- Không trả stack trace.
- Không trả SQL/internal exception ra ngoài.

Exception design

- Có base exception.
- Có business exception.
- Có not found exception.
- Có external service exception.
- Có validation handling.
- Có security exception handling.
- Có fallback `Exception.class`.

Logging

- Log có traceId.
- Log phân biệt INFO/WARN/ERROR.
- Không log sensitive data.
- Lỗi 500 có stack trace.
- Lỗi business expected không spam ERROR.

Security

- 401 xử lý bằng AuthenticationEntryPoint.
- 403 xử lý bằng AccessDeniedHandler.
- Không expose lý do token invalid quá chi tiết.
- Không expose username tồn tại hay không trong login.

External service

- Timeout rõ ràng.
- Retry có kiểm soát.
- Circuit breaker nếu service không ổn định.
- Không retry vô hạn.
- Có fallback nếu phù hợp.
- Có correlationId khi gọi service khác.

Database

- Duplicate key map sang 409.
- Optimistic lock map sang 409.
- DB connection error map sang 500.
- Không expose constraint name nếu không cần.

Async / Event

- Async exception được log.
- Kafka/RabbitMQ có retry và DLT.
- Consumer idempotent.
- Job có execution log.

27. Câu trả lời phỏng vấn mẫu

Trong dự án production, em không để exception trả trực tiếp về client. Em thiết kế một cơ chế global exception handling bằng `@RestControllerAdvice`, trong đó tất cả lỗi được chuẩn hóa về một format chung gồm `timestamp`, `status`, `errorCode`, `message`, `path` và `traceId`.

Em chia exception thành nhiều nhóm. Nhóm business exception dùng cho các lỗi nghiệp vụ như order đã thanh toán, số dư không đủ, trạng thái không hợp lệ. Nhóm not found dùng cho các resource không tồn tại. Nhóm validation xử lý lỗi từ `@Valid`, `MethodArgumentNotValidException`,

`ConstraintViolationException`. Nhóm external service dùng cho lỗi khi gọi payment, shipping, email hoặc third-party API. Ngoài ra còn xử lý riêng lỗi security như 401 và 403 bằng `AuthenticationEntryPoint` và `AccessDeniedHandler`, vì các lỗi này thường xảy ra trước khi request vào controller.

Về logging, em không log tất cả exception ở mức ERROR. Với validation error hoặc business error expected thì chỉ log INFO hoặc WARN. Với lỗi database, external timeout hoặc unexpected exception thì log ERROR kèm stack trace. Tất cả log đều có `traceId` thông qua MDC để khi production incident có thể tìm toàn bộ log của một request.

Một điểm quan trọng là response trả cho client phải an toàn, không expose stack trace, SQL error, internal URL hoặc thông tin nhạy cảm. Message nội bộ có thể log ở server, còn message trả ra ngoài phải được kiểm soát. Với external service, em phân biệt timeout, service unavailable và bad gateway để map sang 504, 503 hoặc 502. Nếu hệ thống dùng Kafka hoặc async processing, em không để exception bị mất mà cấu hình retry, dead letter topic và idempotency để đảm bảo message không bị xử lý sai hoặc lặp.

Tóm lại, exception handling chuẩn production cần đảm bảo ba yếu tố: response nhất quán cho client, log đủ thông tin để vận hành, và không làm lộ thông tin nội bộ của hệ thống.

28. Flow tổng quan

```
Client
|
| HTTP Request
v
Controller
|
| @Valid validation
v
Service Layer
|
| Business rule / repository / external call
v
Exception thrown
|
v
GlobalExceptionHandler
|
| Map exception -> ErrorCode -> HTTP Status
| Add traceId
| Build safe response
v
Client receives standardized error response
```

29. Kết luận

Một exception handling chuẩn production trong Spring Boot nên được thiết kế như một phần của kiến trúc hệ thống, không phải xử lý tạm thời bằng `try-catch`.

Thiết kế tốt cần có:

- `ErrorCode` thống nhất.
- Custom exception rõ ràng.
- `GlobalExceptionHandler`.
- Response lỗi nhất quán.
- `TraceId` trong response và log.
- Không expose lỗi nội bộ.
- Logging đúng level.
- Xử lý riêng security, validation, database, external service.
- Có chiến lược cho async, Kafka, scheduled job.

Cách làm này giúp backend dễ maintain, frontend dễ xử lý lỗi, QA dễ test case, và production team dễ debug khi có incident.